B.TECH CAPSTONE PROJECT · GROUP NO. PIET CSE NON ASMP - 8

COLLABRIX

A Team-First Freelance Collaboration Platform

STUDENT TEAM MEMBERS & ENROLLMENT IDS

- **Jaghatha Uday Prakash** [2303031240447]
- **Boddu Venkata Sai Kowshik** [2303031241591]
- **Katreddi Hima Nagendra** [2303031240520]
- **Immadi Venkata Rama Chaitanya Vardhan** [2303031240436]

PROJECT GUIDANCE & INSTITUTION

Project Guide:	Assistant Prof. Barkha Makwana
Degree Award:	Bachelor of Technology (B.Tech)
Department:	Computer Science & Engineering (Spl. in AI)
Faculty:	PIET · Faculty of Engineering & Technology
Institution:	Parul University, Vadodara · October 2026

SPEAKER SCRIPT / PRESENTATION NOTES:

Good morning respected guide Assistant Professor Barkha Makwana, evaluators, and colleagues. Welcome to our final year capstone presentation on CollabriX: A Team-First Freelance Collaboration Platform. My name is Jaghatha Uday Prakash, presenting alongside my team members Boddu Venkata Sai Kowshik, Katreddi Hima Nagendra, and Immadi Venkata Rama Chaitanya Vardhan from the Department of Computer Science and Engineering with specialization in AI at Parul University. Over the next 25 slides, we will cover the platform's motivation, 12-table relational architecture, PostgreSQL Row-Level Security, core synergy & leaderboard algorithms, live application screens, and empirical validation with 104 passing database checks and top Lighthouse scores.

EXECUTIVE AGENDA

Presentation Roadmap

01 Phase 1: Foundation

Slides 3 – 7

- **Industry Problem:** Failure of solitary gig model
- **CollabriX Solution:** Team-first collaboration paradigm
- **Project Objectives:** 5 core architectural design goals
- **Literature Survey:** Academic foundations & research
- **Market Comparison:** CollabriX vs. Upwork, Fiverr, etc.

02 Phase 2: Architecture

Slides 8 – 11

- **Technology Stack:** React 18, Vite, TypeScript, Supabase
- **System Architecture:** 4-Tier layered system blueprint
- **Database Schema:** 12-Table relational data model
- **5 Extension Tables:** Batches, notifications, invites, settings

03 Phase 3: Algorithms

Slides 12 – 19

- **Connections Protocol:** LinkedIn-style peer networking
- **Synergy Algorithm:** Algorithm 1: Skill coverage ratio
- **Project Lifecycle:** State machine & gated assigning
- **Security & RLS:** PostgreSQL Row-Level Security
- **Rating Integrity:** Completion-gated two-way trust rules
- **Leaderboard Math:** Algorithm 2: Verifiable delivery math

04 Phase 4: Verification

Slides 20 – 25

- **UI Screenshots:** Developer, team & client interfaces
- **UI Screenshots:** Notifications & project lifecycle
- **Verification Suite:** 104 DB checks, 18 browser tests
- **Quality Metrics:** 0 TS errors, ESLint 49→2, Lighthouse
- **Future Roadmap:** AI matching, WebRTC, Stripe escrow
- **Conclusion & Q&A;:** Key takeaways & open floor

SPEAKER SCRIPT / PRESENTATION NOTES:

Here is our presentation roadmap structured into four distinct phases across 25 slides. In Phase 1, we establish the problem space: why individual-only gig platforms fail cross-functional software projects, and benchmark CollabriX against Upwork, Fiverr, Freelancer, and Toptal. In Phase 2, we detail our 4-tier architecture and 12-table relational database schema. In Phase 3, we dissect our core mathematical workflows: team synergy calculations, project lifecycle state machines, Row-Level Security policies, and our open leaderboard formula. Finally, in Phase 4, we showcase live application screenshots, review our 104 automated database test cases and Lighthouse performance audits, and discuss future enhancements.

INDUSTRY CHALLENGE

The Problem: Structural Gaps in Gig Marketplaces

CORE REALITY: Over 68% of commercial software projects require 2+ complementary skills (Front-end, Back-end, Cloud, UI/UX), yet mainstream freelance marketplaces force a rigid 'one-client to one-freelancer' bidding model that isolates talent.

01 Individual-Only Model

Zero native peer teaming

Upwork and Fiverr treat developers as isolated solitary sellers. There is no first-class mechanism for developers to discover peers and bid collaboratively as a cohesive team.

02 Severe Bidding Fatigue

Wasteful unpaid bidding cycles

Clients are overwhelmed by hundreds of generic proposals. Freelancers spend hours writing unpaid bids instead of building software, leading to low hit-rates and frustration.

03 Weak Rating Integrity

Rampant 5-star review inflation

Reviews are decoupled from completed deliverables. Feedback inflation degrades rating signals over time, making star ratings unreliable indicators of actual engineering competence.

04 Communication Silos

Lost context across external apps

Collaboration inevitably spills over to external apps like WhatsApp, Discord, or email. The platform loses project context, auditability, and dispute resolution clarity.

05 Cold-Start Barrier

Invisible newcomers

New entrants without an established review history are invisible in search algorithms, regardless of actual technical skill, creating an insurmountable barrier for student talent.

SPEAKER SCRIPT / PRESENTATION NOTES:

Turning to Slide 3, we analyze the structural breakdown in contemporary freelance platforms. The fundamental flaw is the individual-only hiring paradigm. While modern software demands multi-disciplinary teams, platforms like Upwork and Fiverr are architected around a single seller profile. This mismatch causes five severe issues: First, developers cannot officially team up on-platform. Second, clients face proposal fatigue while developers waste countless unpaid hours submitting blind bids. Third, reputation systems suffer from systemic star inflation because reviews are not cryptographically or relationally tied to completed deliverables. Fourth, communication splinters across WhatsApp and Slack. And fifth, early-career and student developers face an impenetrable cold-start barrier.

CORE INNOVATION

The Solution: CollabriX Team-First Paradigm

■ The Paradigm Shift

From Solitary Bidding to Collaborative Delivery

• **Team-First Architecture:** Collaboration is the core data primitive rather than a cosmetic afterthought. Developers assemble complementary teams directly on-platform.

• **Multi-Regional Discovery:** Discover peers by continent, sub-region, country, and tech stack to form time-zone aligned development squads.

• **Dynamic Team Batches:** Group up to 10 verified connections into reusable team units with distinct project roles and automated skill coverage calculations.

• **Dual Role Workspaces:** Dedicated client dashboard for posting work and managing teams, paired with a rich developer hub for networking and jobs.

■■ Four Architectural Pillars

Engineered for Integrity & High Performance

• **1. Synergy Engine:** Mathematically computes skill complementarity against domain benchmarks, identifying missing technology gaps before bidding.

• **2. Relational Governance:** PostgreSQL triggers and constraints prevent invalid transitions, fake reviews, and unauthorized assignments at the database level.

• **3. Zero-Baseline Reputation:** Public leaderboard computed exclusively from completed client contracts, eliminating synthetic baseline ratings and bias.

• **4. Realtime Synchronization:** WebSocket-driven peer chat and automated notifications without polling overhead or communication leakage.

KEY VALUE PROPOSITION: CollabriX enables student and early-career developers to pool complementary skills, compete for larger and higher-paying commercial software contracts, and build an unforgeable verified track record on an open platform.

SPEAKER SCRIPT / PRESENTATION NOTES:

On Slide 4, we introduce the CollabriX solution. The name combines 'Collaboration' with 'X'—representing the multiplier effect of team synergy. Instead of treating developers as isolated competitors in a race to the bottom, CollabriX provides a first-class collaborative workspace. Our system is founded on four architectural pillars: First, dynamic team batching with our synergy scoring engine, which evaluates a squad's tech stack against project domains. Second, database-enforced relational governance using PostgreSQL Row-Level Security, guaranteeing that no state or rating can be forged. Third, a transparent zero-baseline leaderboard that rewards verified delivery rather than opaque commercial algorithms. And fourth, sub-second WebSocket communication keeping teams aligned without external tool fragmentation.

ENGINEERING TARGETS

Project Objectives & Design Goals

01

Dual Role-Based Workspaces — Implement strict architectural and UI separation between Developer and Client experiences, guaranteeing tailored navigation, role-specific tooling, and secure data isolation.

02

Automated Synergy Matching — Design and implement Algorithm 1 to match combined team member skills against standardized industry domain stacks, dynamically flagging missing technology gaps.

03

Database-Enforced Lifecycle — Construct a deterministic 4-state project machine (Open → In Progress → Completed / Cancelled) where project assignment and ratings require relational proof of prior stages.

04

Meritocratic Leaderboard Ranking — Formulate Algorithm 2 in PostgreSQL RPC without arbitrary baseline scores, ranking developers deterministically based strictly on verified delivery volume and client star ratings.

05

Sub-Second Event-Driven Collaboration — Deliver instantaneous peer-to-peer messaging and trigger-generated notifications using Supabase Realtime WebSockets, completely eliminating client-side polling latency.

SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 5 outlines our five primary engineering objectives. First, establish dual role-based workspaces for developers and clients with distinct permissions and interfaces. Second, build an automated team formation engine powered by Algorithm 1 that evaluates skill complementarity and highlights technology gaps. Third, govern project lifecycles at the database layer so that assignments and reviews cannot bypass verified states. Fourth, design a meritocratic leaderboard algorithm implemented directly as a PostgreSQL stored procedure, removing opaque ranking biases. And fifth, integrate sub-second WebSocket synchronization for real-time messaging and trigger-driven notifications.

Literature Survey: Theoretical Foundations

[1] Agrawal, Horton, Lacetera & Lyons (2015)

Digitization and the Contract Labor Market

• **Finding:** Demonstrated that online labor markets exhibit severe spatial and informational frictions; geographic clustering and time-zone alignment strongly dictate hiring success.

• **Adoption:** CollabriX Implication: Implemented hierarchical regional filtering (Continent → Sub-region → Country) allowing developers to discover peers in compatible working time-zones.

[2] Filippas, Horton & Golden (2020)

Reputation Inflation in the Gig Economy

• **Finding:** Disclosed that conventional 5-star rating mechanisms suffer extreme upward grade inflation, severely diminishing their signaling power for clients.

• **Adoption:** CollabriX Implication: Implemented completion-gated two-way reviews locked behind database triggers, ensuring ratings only originate from verified completed contracts.

[3] Retelny et al. (2014) & Valentine et al. (2017)

Flash Teams & Flash Organizations

• **Finding:** Proved that crowd workers dynamically assembled into structured teams can accomplish complex, open-ended engineering goals when provided with role structures.

• **Adoption:** CollabriX Implication: Designed our native Team Batch engine, providing project domain role assignments and automated skill complementarity calculations.

[4] Hannák et al. (2017)

Bias in Online Freelance Marketplaces

• **Finding:** Found that commercial algorithmic ranking engines introduce systematic visibility barriers against newcomers and non-traditional freelancers.

• **Adoption:** CollabriX Implication: Engineered a transparent, deterministic leaderboard formula based on volume of delivered work, eliminating opaque corporate ranking algorithms.

SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 6 reviews the academic literature informing our engineering decisions, drawn from Chapter 2 of our project report. Studies by Agrawal et al. highlighted that geographic and time-zone frictions heavily degrade solo freelancing; we resolved this with hierarchical region-based discovery. Filippas and colleagues documented the crisis of rating inflation in the gig economy, where almost every worker is rated 5 stars; we mitigated this by locking ratings behind completion-gated relational triggers. Seminal research on Flash Teams from Stanford by Retelny and Valentine demonstrated that online crowd workers succeed on complex software tasks when platforms provide structural role scaffolding—the exact premise of our Team Formation Engine. Finally, Hannák et al. exposed algorithmic ranking biases against newcomers; we solved this with an open, mathematically deterministic ranking formula.

COMPETITIVE BENCHMARK

Comparative Analysis: CollabriX vs. Market Leaders

Aspect / Dimension	Upwork	Fiverr	Freelancer	Toptal	CollabriX (Ours)
Hiring Model	Job post + proposals	Fixed gigs / packages	Bidding & contests	Curated 3% match	**Profiles + Teams**
Peer Discovery	Limited / absent	None	Limited	None	**Full Search & Connect**
Form Teams on Platform	Agencies only	No	No	Manual staff match	**Yes (Team Batches)**
Team Synergy Scoring	None	None	None	Manual recruiter	**Algorithmic (Alg. 1)**
Rating Governance	Platform policy	Review-based	Contest feedback	Internal scoring	**PostgreSQL RLS Triggers**
Client Commission	Up to 10%	Up to 20%	10% + fees	High corporate premium	**0% (Open Academic)**
Ranking Transparency	Opaque JSS score	Proprietary algorithm	Bid amount priority	Internal discretion	**Public Math Formula**

SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 7 benchmark-tests CollabriX against industry titans Upwork, Fiverr, Freelancer, and Toptal, reproducing Table 2.1 from our report. Notice the structural divergence: on Upwork, teaming is restricted solely to formal legal agencies; on Fiverr and Freelancer, it is nonexistent. Toptal coordinates teams only through high-cost internal recruiters. CollabriX is the only platform that offers dynamic, first-class peer discovery, native Team Batches with algorithmic synergy evaluation, database-level rating integrity enforced by PostgreSQL RLS, and a public, mathematically verifiable leaderboard.

Technology Stack & Technical Justification

■ Frontend Core

Reactive & Type-Safe

• **React 18:** Component-driven reactive UI architecture with efficient virtual DOM reconciliation.

• **TypeScript 5:** Strict static type system synchronized directly with generated Supabase database types.

• **Vite Bundler:** Lightning-fast Hot Module Replacement and optimized Rollup production builds (< 2s).

■ UI & Styling

Design Tokens & Access

• **Tailwind CSS:** Utility-first design tokens with customized teal/slate palettes and responsive breakpoints.

• **shadcn/ui & Radix:** Accessible, unstyled headless primitives (Dialogs, Dropdowns, Badges, Tabs, Tooltips).

• **Lucide Icons:** Crisp, lightweight SVG icons consistent across all developer and client modules.

■ State & Client Data

Server-State Caching

• **TanStack Query v5:** Declarative server-state caching, automatic background invalidation, and optimistic updates.

• **Zod Schema Engine:** Strict runtime form and payload schema validation preventing malformed API requests.

• **React Router 6:** Protected declarative routing with client-side role guards and session restoration.

■■ Backend & Database

BaaS & Relational Security

• **Supabase BaaS:** Managed GoTrue authentication, PostgREST API generation, and Realtime WebSocket engine.

• **PostgreSQL 16:** Enterprise relational database with ACID compliance, custom ENUMs, triggers, and RPCs.

• **Row-Level Security:** Database-level policy enforcement isolating multi-tenant rows via auth.uid() tokens.

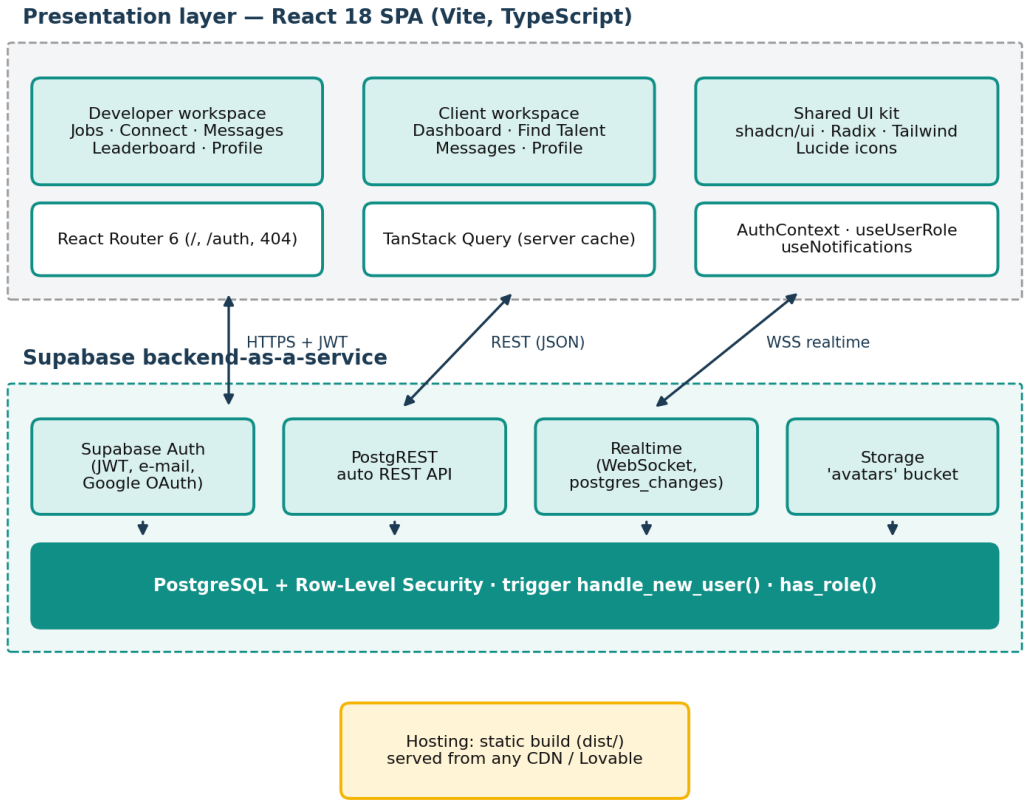
SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 8 outlines the four layers of our technology stack, justified in Table 3.1 of our report. On the client, React 18 and Vite deliver sub-2-second build times and rapid HMR, while TypeScript enforces strict type consistency with our Supabase database schema. Our UI pairs Tailwind CSS with accessible shadcn/ui primitives. For client-side caching and state synchronization, TanStack Query provides optimistic updates and automatic cache invalidation. Crucially, instead of writing a redundant Express.js backend layer that introduces latency and vulnerability surfaces, we leverage Supabase with PostgreSQL 16. Business rules, validation triggers, and access control policies live directly inside the database, broadcasting instant state updates via Supabase Realtime WebSockets.

System Architecture: 4-Tier Layered Blueprint

■ Architectural Blueprint

Figure 3.3 from Project Report



■■ Tier Responsibilities & Flow

End-to-End Separation of Concerns

• **Tier 1: Presentation Layer:** Responsive mobile-first React UI, bottom navigation bar for mobile, header notifications bell, and role-specific screens.

• **Tier 2: Client State Layer:** Custom domain hooks (useAuth, useUserRole, useProjects, useNotificationFeed) decoupling business logic from UI components via TanStack Query.

• **Tier 3: Backend-as-a-Service:** Supabase GoTrue service for JWT session management, PostgREST for autogenerated REST endpoints, and WebSocket channels for realtime replication.

• **Tier 4: Relational Security Layer:** PostgreSQL 16 relational engine enforcing Row-Level Security (RLS) policies, security-definer RPC functions, and automated event triggers.

SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 9 displays the four-tier architectural blueprint of CollabriX, representing Figure 3.3 from our documentation. At the top is the Presentation Layer, built with responsive mobile-first React components and accessible Radix primitives. Directly below it, the Client State Layer isolates server communication into modular React hooks like useAuth, useProjects, and useNotificationFeed, ensuring clean separation of concerns and eliminating component-level side effects. The third tier is the Backend-as-a-Service layer powered by Supabase, providing authenticated PostgREST queries and WebSocket subscriptions. The foundational tier is PostgreSQL 16. By anchoring authorization, state machines, and notification creation inside the database engine, CollabriX guarantees zero unauthorized operations even if a client attempts to bypass the web frontend.

Database Architecture: 12 Relational Tables

■ 7 Core Foundation Tables

Original sprint schema managing identity & jobs

- **profiles:** Public identity: name, avatar, bio, developer type, skills, region (PK = auth.users.id CASCADE).
- **user_roles:** Authorisation roles: developer or client enum with UNIQUE(user_id, role) constraint.
- **connections:** LinkedIn-style peer requests between developers: sender, receiver, status (pending/accepted).
- **messages:** Realtime chat messages inside accepted connections; enabled for WebSocket replication.
- **projects:** Client project postings: title, scope, budget, complexity (Easy/Med/Hard), status lifecycle.
- **project_ratings:** Multi-dimensional project feedback (1–5 stars) submitted upon verified completion.
- **user_ratings:** Peer-to-peer user evaluation (1–5 stars) linked to completed project contracts.

■ 5 Collaboration Extension Tables

Added in final sprint for teams & governance

- **team_batches:** Saved developer squads: 1–10 members, accepted connections only, synergy score.
- **notifications:** System-generated audit feed; 10 event types; rows created solely by DB triggers.
- **project_invitations:** Client-to-developer formal invitations; UNIQUE(project, dev); developer accepts first.
- **user_settings:** User privacy: leaderboard visibility toggle, email/in-app notification preferences.
- **support_tickets:** Platform helpdesk: auto-generated ticket numbering with subject/description length CHECKs.

DATABASE INTEGRITY SUMMARY: All 12 tables reside in PostgreSQL's public schema. Every profile shares its primary key with Supabase's auth.users table, making auth.uid() directly comparable in Row-Level Security policies. Referential cascades guarantee zero orphaned records upon user deletion.

SPEAKER SCRIPT / PRESENTATION NOTES:

Turning to Slide 10, CollabriX is built upon twelve relational tables in the public PostgreSQL schema, detailed in Table 3.10 of our report. The architecture is bifurcated into two groups: The seven core tables manage user profiles, role enforcement, peer connections, realtime messages, project listings, and dual-party ratings. The collaboration extension introduced five new tables: team_batches for squad composition, notifications for trigger-driven alerts, project_invitations for client outreach, user_settings for privacy control, and support_tickets for issue reporting. Every profile primary key links directly to auth.users with ON DELETE CASCADE, allowing Row-Level Security policies to evaluate auth.uid() efficiently.

Database Design: The 5 New Tables Deep-Dive

■ team_batches

Collaborative Squads

• **Schema:** id, owner_id, name, domain, members (JSONB), synergy_score, created_at

• **Trigger Rule:** BEFORE INSERT trigger verifies 1–10 members, confirms all members are accepted connections, and rejects owner self-inclusion.

• **Integrity:** Batches saved locally in older client versions are imported automatically upon first sign-in.

■ notifications

Trigger-Driven Feed

• **Schema:** id, user_id, actor_id, type (10 enums), title, body, link, is_read, created_at

• **Security:** Direct INSERT by client/developer roles is strictly prohibited; rows are inserted solely by PostgreSQL AFTER triggers.

• **Preference:** Trigger inspects user_settings before creating notification; suppresses muted alerts.

■ project_invitations

Gated Assignment

• **Schema:** id, project_id, developer_id, client_id, status (pending/accepted/declined/cancelled)

• **Constraint:** UNIQUE(project_id, developer_id) prevents spamming; only project owner can invite.

• **Assignment Rule:** Client cannot assign developer until status transitions to 'accepted'.

■■ user_settings

Privacy & Preferences

• **Schema:** user_id (PK = auth.uid), in_app_notifs, email_notifs, show_on_leaderboard, theme

• **Privacy:** Developers can toggle show_on_leaderboard to false, instantly hiding from public ranking.

• **RLS Isolation:** Users can view and update exclusively their own settings record.

■ support_tickets

Helpdesk Submissions

• **Schema:** id, ticket_number (TICK-XXXX sequence), user_id, category, subject, description, status

• **CHECK Rules:** Subject must be 5–150 chars; description must be 20–5000 chars; category in allowed list.

• **State Flow:** Must start in 'open' status; clients cannot arbitrarily edit status once submitted.

SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 11 delves into the precise technical schema and constraints of our five new extension tables. In team_batches, a database BEFORE trigger validates that every member is an accepted connection of the creator and limits size to 10. In notifications, client-side INSERT is completely disabled; records are written solely by database triggers responding to business events. In project_invitations, composite unique constraints prevent invitation spam, and project assignment requires the developer to accept first. In user_settings, developers are granted privacy sovereignty, allowing them to opt out of the public leaderboard. Finally, support_tickets enforces text length CHECK constraints and auto-increments public ticket identifiers.

Connections & Peer Discovery Protocol

■ Step 1: Multi-Facet Discovery

Region & Skill Filtering

Developers filter peers across Continent → Sub-region → Country, combined with specific technology tags to find compatible partners.

■ Step 2: Request Dispatch

Personalized Introduction

Sender transmits a connection request accompanied by an introductory pitch; row inserted with status 'pending'; DB trigger alerts receiver.

■ Step 3: Receiver Evaluation

Accept or Decline Action

Receiver inspects sender profile, ratings, and shared skills. Accepting transitions status to 'accepted', while declining updates to 'rejected'.

■ Step 4: Realtime Unlocked

WebSocket Chat & Teaming

Accepted status automatically activates real-time messaging thread via Supabase Realtime; receiver can now be added to Team Batches.

■ Relationship State Machine

Table 3.18 from Report

- **None / Stranger:** UI offers 'Connect' button; profile skills and ratings are publicly viewable.
- **Pending (Sent):** UI displays 'Pending' badge; awaiting receiver confirmation; no messaging allowed.
- **Pending (Received):** UI displays 'Accept' and 'Decline' actions with real-time notification count.
- **Accepted:** UI unlocks 'Message' button and allows member selection in Team Formation Engine.

■ Unread Tracking via Security-Definer RPC

Solving the RLS Read Paradox

- **The Paradox:** Strict RLS permitted only message senders to UPDATE their messages, preventing receivers from marking messages as read.
- **The RPC Solution:** Engineered mark_messages_read(connection) as a security-definer PostgreSQL stored procedure.
- **Dynamic Badging:** When recipient opens a chat thread, unread counters clear instantly across navigation bar and desktop header.

SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 12 explains our Connections and Peer Discovery protocol. Developers discover peers using our multi-tier regional filter and technical skill tags. The connection follows a four-step request-accept lifecycle similar to LinkedIn. Crucially, messaging and team formation are strictly gated by the 'accepted' status. We also solved a notable database permissions paradox: under strict RLS, users could not update someone else's message row to set is_read=true. To resolve this securely, we built the mark_messages_read security-definer function, which validates that the caller is indeed the recipient and marks messages read, dynamically clearing navigation unread counters in real time.

Team Formation Engine & Synergy Algorithm

■ Algorithm 1: Synergy Score Calculation

Mathematical Formulation

• **Domain Benchmark Stack (R):** Predefined industry requirements (e.g. Fullstack Web: React, TypeScript, Node.js, PostgreSQL, UI/UX, AWS).

• **Combined Squad Stack (S):** Union of all selected members' verified skills: S = ∪ { skills(m) : m ∈ M }.

• **Skill Coverage Ratio:** Ratio = |Covered Skills| / |Required Benchmark Stack|.

• **Team Size Bonus:** Bonus = min(10 × |Members|, 30) — awards points for balanced team capacity.

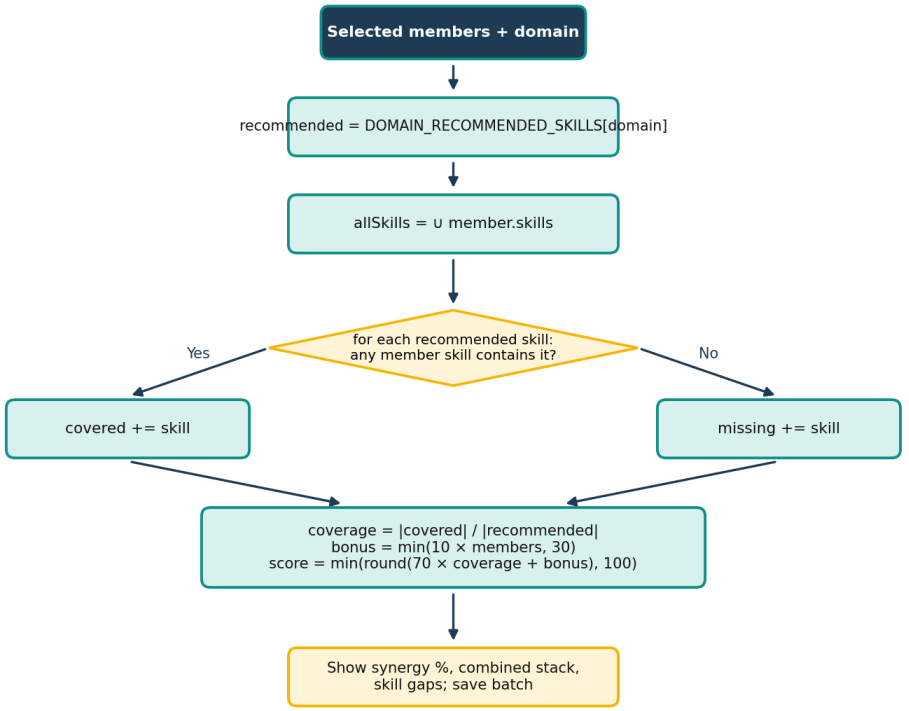
• **Synergy Score Formula:** Score = min(round(70 × Ratio + Bonus), 100); if |M| = 0 then Score = 0.

• **Worked Example:** A 2-member team covering 4 of 6 skills: round(70 × 4/6 + 20) = 47 + 20 = 67%.

• **Proactive Skill Gaps Alert:** The UI flags missing skills (e.g. AWS, UI/UX Design), guiding teams to invite complementary connections before bidding.

■ Algorithm 1 Flowchart

Figure 3.14 from Project Report



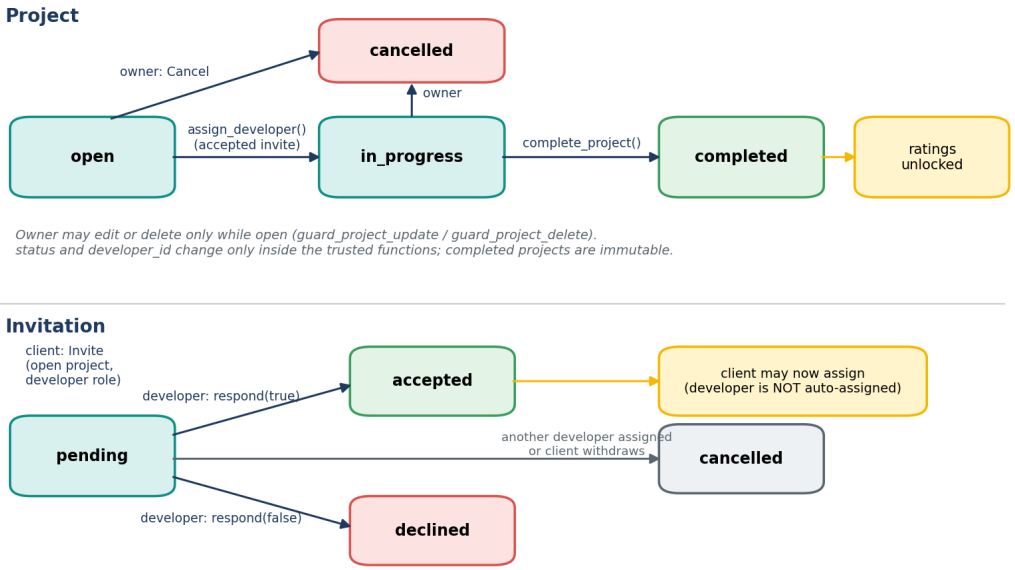
SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 13 details our proprietary Team Formation Engine and Algorithm 1, illustrated in Figure 3.14. When assembling a team batch, the creator selects a target project domain, such as Fullstack Web App, Mobile Engineering, or Cloud Infrastructure. The engine computes the mathematical union of all members' skills and evaluates it against the domain's recommended technology benchmark. The formula awards up to 70 points for skill coverage and up to 30 bonus points for team capacity, capped at 100%. For example, a two-developer squad covering four of six required stack technologies achieves a 67% synergy score. Crucially, the UI highlights the remaining two missing skills as skill gaps, prompting the squad to recruit a complementary peer.

Project Lifecycle Management

■ Project & Invitation Lifecycles

Figure 3.11 from Project Report



■ Lifecycle Rules & Algorithm 3

Deterministic State Progression

• **State 1: Open:** Created exclusively by authenticated clients; developers browse listings; client sends invitations to candidates or team batches.

• **State 2: In Progress:** Transitioned via Algorithm 3: client issues invitation → developer explicitly accepts → client confirms assignment.

• **State 3: Completed:** Client marks deliverables approved. This transition unlocks two-way rating eligibility in the database.

• **State 4: Cancelled:** Client aborts engagement prior to completion; ratings are permanently disabled to prevent review bombing.

• **Client-Only Posting Trigger:** PostgreSQL BEFORE trigger rejects any project INSERT where the author does not hold the 'client' role.

• **Anti-Circumvention Rule:** A client cannot directly assign a developer without their explicit in-app consent via accepted project invitation.

SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 14 explains our Project Lifecycle and Algorithm 3, depicted in Figure 3.11. In CollabriX, project progression is governed as a deterministic four-stage finite state machine: Open, In Progress, Completed, and Cancelled. To ensure developer autonomy, a client cannot unilaterally assign a project to a freelancer. Instead, Algorithm 3 enforces an invitation-gated assignment protocol: the client sends an invitation, the developer reviews and accepts it, and only then does the database permit the client to assign the project. Once marked 'Completed' by the client, the project unlocks mutual rating capabilities. If a project is cancelled, ratings are disabled permanently to prevent retaliatory negative reviews.

Security Model: PostgreSQL Row-Level Security

■■ Row-Level Security (RLS)

Policy Enforcement on All 12 Tables

• **Tenant Isolation:** auth.uid() = user_id filter ensures private profile settings, notifications, and batches are visible only to owners.

• **Role Verification:** has_role(auth.uid(), 'client') ensures developers cannot post projects or forge client invitations.

• **Zero Anonymous Writes:** Unauthenticated requests are strictly restricted to reading public profiles and open projects.

■ Security-Definer RPCs

Eliminating Infinite RLS Recursion

• **The Challenge:** Cross-table RLS policies evaluating user_roles or connections trigger infinite recursive query loops in PostgreSQL.

• **The Fix:** Security-definer helper functions execute with owner privilege while asserting caller session token.

• **Core Functions:** has_role(), mark_messages_read(), get_leaderboard(), and create_notification().

■ Threat Mitigation Matrix

Table 3.16 Threat Modeling

• **Privilege Escalation:** Roles stored in dedicated user_roles table rather than mutable profile columns; prevents self-promotion.

• **Data Snooping:** Messages and chat threads restricted to accepted connection pairs via composite RLS policies.

• **Review Forgery:** Rating inserts verified by checking project completed status and counterpart identity.

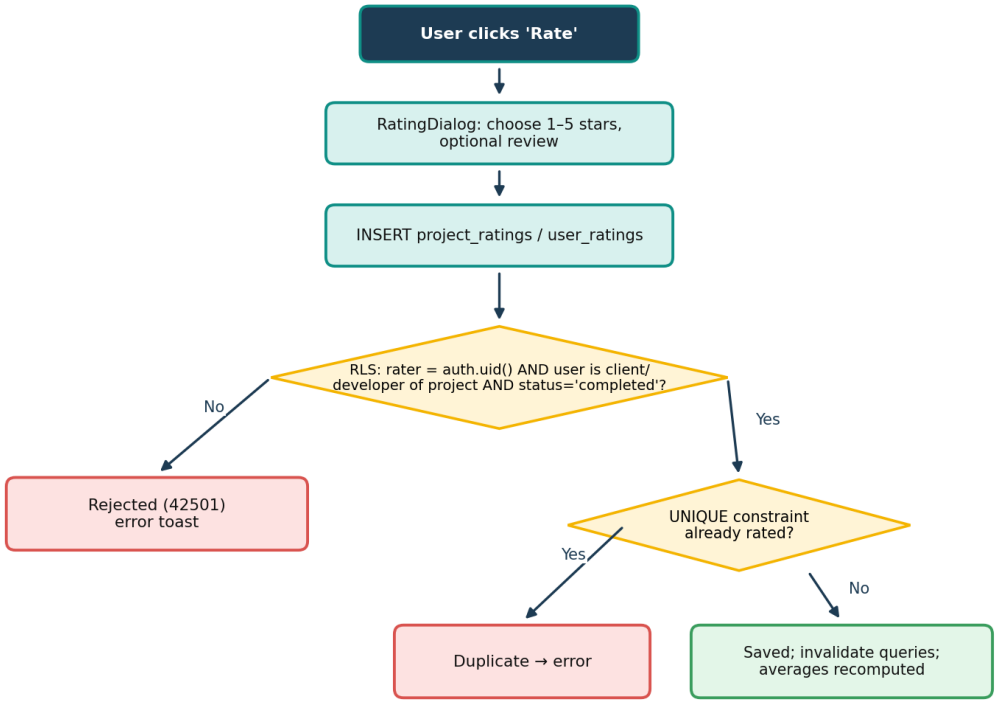
SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 15 presents our database security architecture. Rather than relying on frontend checks that can be circumvented via API inspection, CollabriX anchors access control directly inside PostgreSQL using Row-Level Security on all 12 tables. To avoid infinite recursion during complex relational joins—such as checking whether a user is a client while evaluating a project policy— we engineered security-definer helper functions that execute with administrative privileges while strictly validating auth.uid(). Our threat mitigation model, detailed in Table 3.16 of the report, prevents privilege escalation by storing roles in a separate table, blocks eavesdropping on messages, and makes review forgery mathematically impossible.

Two-Way Ratings & Reputation Integrity

■ Rating Submission & Validation

Figure 3.15 from Project Report



■■ The 5 Pillars of Rating Integrity

Preventing Grade Inflation & Fake Reviews

• **1. Completion-Gated Submission:** RLS policies strictly assert projects.status = 'completed'. Ratings cannot be submitted for open or in-progress projects.

• **2. Counterpart Verification:** A client can only rate their assigned developer; a developer can only rate their project client. Third-party review insertion is blocked.

• **3. Exactly One Rating (Idempotency):** UNIQUE(project_id, rater_id) constraints ensure neither participant can submit multiple reviews or artificially manipulate scores.

• **4. Dual-Perspective Evaluation:** Captures both Project-level satisfaction (project_ratings) and Peer/Client interpersonal feedback (user_ratings).

• **5. Immutability Guarantee:** Once submitted, ratings cannot be modified or deleted via client APIs, preventing coercive renegotiation or retaliatory score alterations.

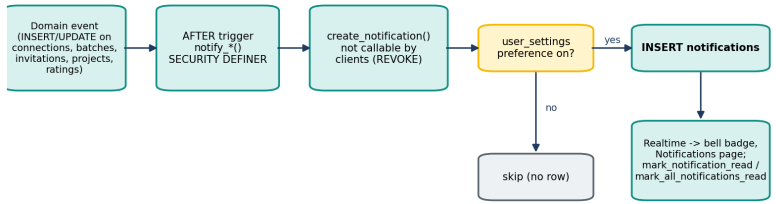
SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 16 details our two-way rating system and reputation integrity rules, illustrated in Figure 3.15. To combat the pervasive rating inflation and fake reviews seen on mainstream platforms, CollabriX enforces five non-negotiable database rules. First, reviews are strictly completion-gated: if a project is not marked 'completed', all rating inserts are rejected with a permission error. Second, only the two contractual counterparts can rate each other. Third, composite unique constraints guarantee exactly one rating per participant per contract. Fourth, we separate project feedback from individual peer evaluations. And fifth, ratings are completely immutable once committed, preventing retaliatory score tampering.

Automated Event-Driven Notification Pipeline

■ Database Trigger to User Interface Pipeline

Figure 3.16 from Project Report



■ 10 Relational Event Triggers

Table 3.21 from Project Report

- **Connections:** request_received, request_accepted
- **Team Batches:** team_batch_added (when added to a collaborative squad)
- **Invitations:** project_invite_received, invite_accepted, invite_declined
- **Projects:** project_assigned, project_completed
- **Reputation:** rating_received (when a counterpart submits feedback)
- **Support:** ticket_status_updated (when support responds to an issue)

■■ Zero Client Forgery & Preferences

Trigger-Enforced Delivery

- **Zero-Forgery Guarantee:** Client roles cannot INSERT or UPDATE notifications directly. Rows are generated exclusively by PostgreSQL AFTER triggers upon genuine database events.
- **Preference Checking:** Trigger queries user_settings before row insertion; if in_app_notifs or specific category is disabled, creation is silently suppressed.
- **Self-Trigger Filtering:** The create_notification function automatically discards actions where actor_id == user_id, preventing self-notification noise.
- **Realtime Delivery:** Supabase Realtime replicates inserts instantly over WebSockets to the header notification bell with sound and unread badge count.

SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 17 depicts our event-driven notification pipeline from Figure 3.16. Unlike conventional architectures where the frontend or API layer pushes notifications manually, CollabriX generates notifications exclusively via PostgreSQL AFTER triggers on our event tables. Whenever a connection is accepted, a teammate is added to a batch, or a project is assigned, our create_notification function executes. It checks the recipient's notification preferences in user_settings, discards actions caused by the user themselves, and inserts the record. Because direct client insertion is prohibited, every notification is cryptographic proof of a verified database event.

Leaderboard Ranking: Mathematical Model

■ Algorithm 2: Ranking Mathematics

Database Function get_leaderboard

• **Mathematical Formula:** Score(d) = 100 × C(d) + 20 × Σ Rating(r), where C(d) is verified completed projects and Rating(r) is client stars (1–5).

• **Completed 5-Star Value:** A single 5-star completed project yields: 100 + (20 × 5) = 200 Points.

• **Zero-Baseline Principle:** No synthetic 4.5 baseline ratings. Developers with zero completed projects remain unranked (Score = 0) until real work is delivered.

• **Deterministic Tie-Breaking:** When scores tie, rank resolves sequentially by: Score ↓, Average Rating ↓, Review Count n ↓, Full Name ↑, User UUID ↑.

• **Dynamic Time Filtering:** get_leaderboard(period_start) dynamically recalculates All-Time, Monthly, and Weekly rankings without stale cache anomalies.

• **Privacy Sovereignty:** Developers can toggle show_on_leaderboard in user_settings, instantly excluding themselves from public competitive rankings.

■ Algorithm 2 Step-by-Step Logic

PostgreSQL Stored Procedure Execution

• **Step 1: Input Parameters:** Accepts optional timestamp t■ for period filtering (NULL denotes all-time career scores).

• **Step 2: Candidate Selection:** Selects all profiles where role = 'developer' and user_settings.show_on_leaderboard = true.

• **Step 3: Relational Aggregation:** Counts completed projects (C) and sums client star ratings (R) completed after t■.

• **Step 4: Score Calculation:** Executes score = 100 × C + 20 × Σ(R) and computes exact arithmetic mean rating.

• **Step 5: Eligibility Filter:** Filters candidates: candidate must have C > 0 or |R| > 0 to enter the leaderboard array.

• **Step 6: Deterministic Sort & Rank:** Sorts array by Score DESC, Avg DESC, Count DESC, Name ASC, and assigns sequential ranks 1...N.

SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 18 details our Leaderboard Ranking Formula, implemented as Algorithm 2 in PostgreSQL. Unlike commercial platforms that employ black-box algorithms prioritizing platform profit or ad spend, CollabriX uses a completely open, meritocratic formula executed inside the database via get_leaderboard. A developer earns 100 points for each verified completed project and 20 points per star awarded by their client. Thus, a flawless 5-star project yields exactly 200 points. We enforce a strict zero-baseline rule: newcomers do not receive arbitrary placeholder scores and only appear on the board after completing real work. Ties are broken deterministically through a multi-tier hierarchy: average rating, review count, and alphabetical name.

Leaderboard in Action: Worked Example & Tiers

■ Table 3.19: Worked Example of Ranking Formula

Deterministic Ordering & Volume Trade-off

Developer	Completed	Client Ratings	Score Formula	Avg	Rank
Chen	3	2, 2, 2	300 + 120 = 420	2.0	**1**
Asha	2	5, 5	200 + 200 = 400	5.0	**2**
Ben	2	5, 3	200 + 160 = 360	4.0	**3**
Dina	1	4	100 + 80 = 180	4.0	**4 (tie)**
Eli	1	4	100 + 80 = 180	4.0	**5 (tie)**
Farah	0	None	0 + 0 = 0	N/A	**Unranked**

DESIGN TRADE-OFF DEMONSTRATED:

- **Work Volume Rewarded:** Chen delivered 3 projects with modest ratings (420 pts), ranking above Asha (400 pts) who completed 2 flawless projects. Real delivered work outweighs small samples.
- **Deterministic Tie-Break:** Dina & Eli match on score and ratings; Dina ranks first alphabetically.
- **Zero Baseline:** Farah has completed no projects, so she remains cleanly unranked.

■ Tier Levels & Badges

Gamification & Recognition

- **Diamond Tier:** 1000+ Points · Veteran elite contractors
- **Gold Tier:** 501–1000 Points · Proven collaborative leads
- **Silver Tier:** 201–500 Points · Consistent verified delivery
- **Bronze Tier:** 1–200 Points · First successful project delivered
- **Project Veteran:** Awarded upon completing 5+ verified projects
- **5-Star Performer:** Awarded for maintaining 4.8+ client average
- **Team Pioneer:** Awarded for forming 3+ multi-member batches
- **Quick Responder:** Awarded for rapid response to invitations

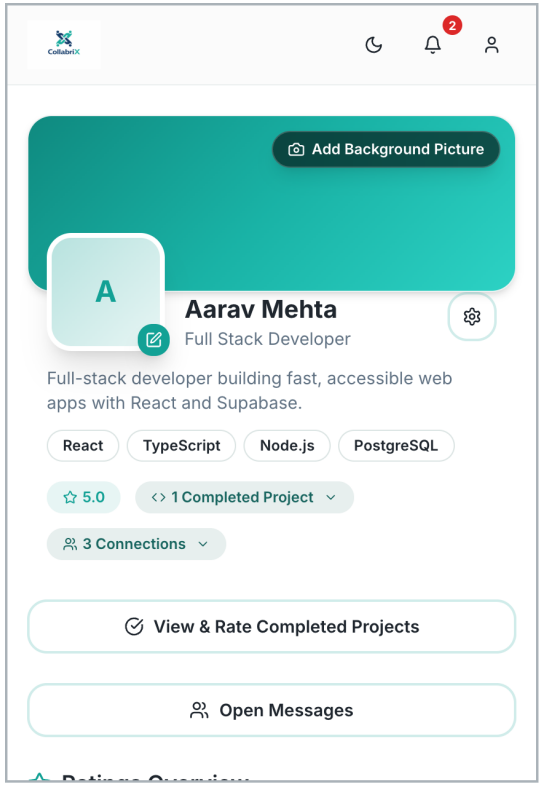
SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 19 walks through Table 3.19 from our report, demonstrating the leaderboard formula in action for six sample developers. Notice the deliberate design trade-off: our formula deliberately rewards the volume of verified delivery. Chen, who delivered three projects with 2-star ratings, scores 420 points and ranks first above Asha, who has two 5-star projects totaling 400 points. This reflects commercial reality: delivering verified client work carries high value. Dina and Eli have identical completed work and ratings; the algorithm breaks the tie deterministically by alphabetical name. Farah has zero completed work, so she remains cleanly unranked rather than receiving an unearned synthetic score. On the right, developer scores unlock four tier milestones and six milestone achievement badges.

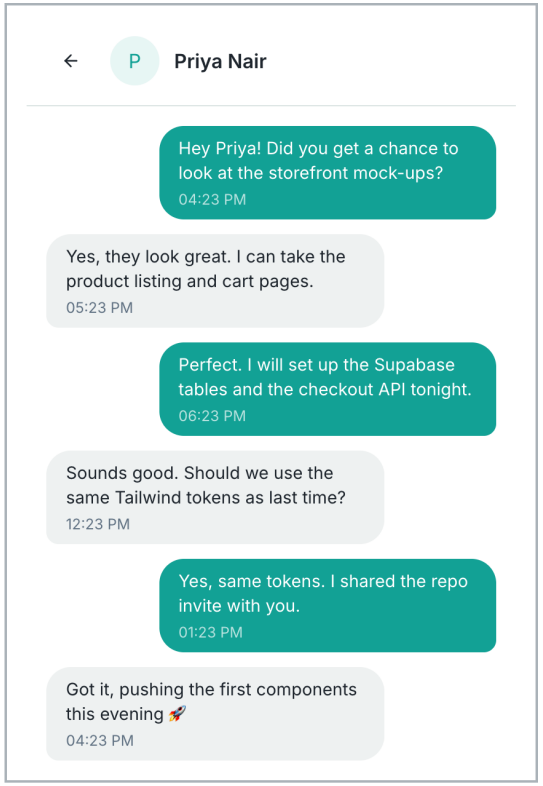
Developer Workspace & Realtime Messaging

Developer Interface & Realtime Chat

Figure 4.4 from Project Report



(a) Developer profile



(b) Real-time chat

Interface Features & Highlights

Live Production Build Validation

- **Screen (a): Developer Profile:** Displays verified technical skills, tier ranking badges, completed project counters, and authentic client rating stars derived directly from SQL joins.
- **Interactive Skill Tags:** Skills are stored as JSONB arrays, enabling instant filtering and matching across project discovery feeds.
- **Screen (b): Real-Time Chat:** One-on-one WebSocket messaging thread between accepted connections, powered by Supabase Realtime channel postgres_changes.
- **Sub-Second Message Sync:** Messages render with zero polling; instant read receipts update when recipient opens the thread.
- **Dynamic Read Badging:** Invokes security-definer function mark_messages_read, dynamically clearing unread counter badges on the navigation bar.

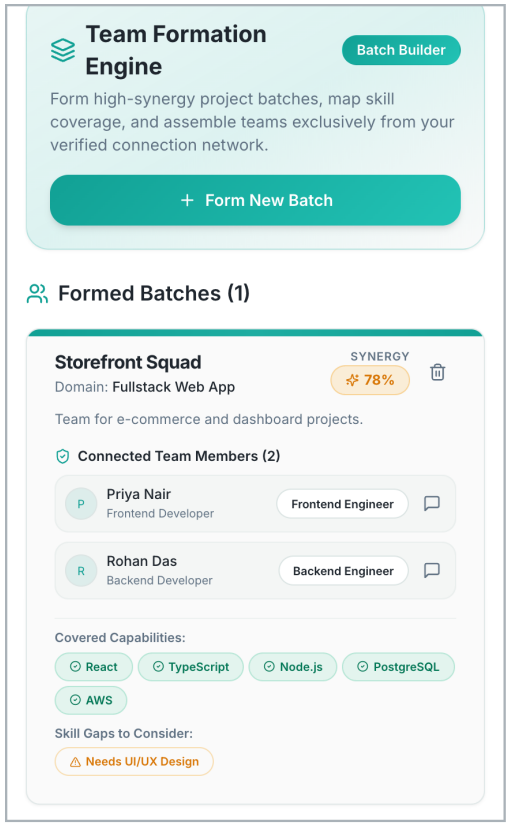
SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 20 showcases live application screenshots from Figure 4.4 of our documentation, captured from our production build. On the left, Screen (a) shows the Developer Profile view, displaying verified skills, level tier, and star ratings computed from database joins. On the right, Screen (b) demonstrates our real-time messaging interface. Chat channels are established exclusively between accepted connections. Messages are delivered instantly via Supabase Realtime WebSocket subscriptions without polling overhead. Opening the conversation automatically invokes mark_messages_read, clearing unread counters across the mobile navigation bar.

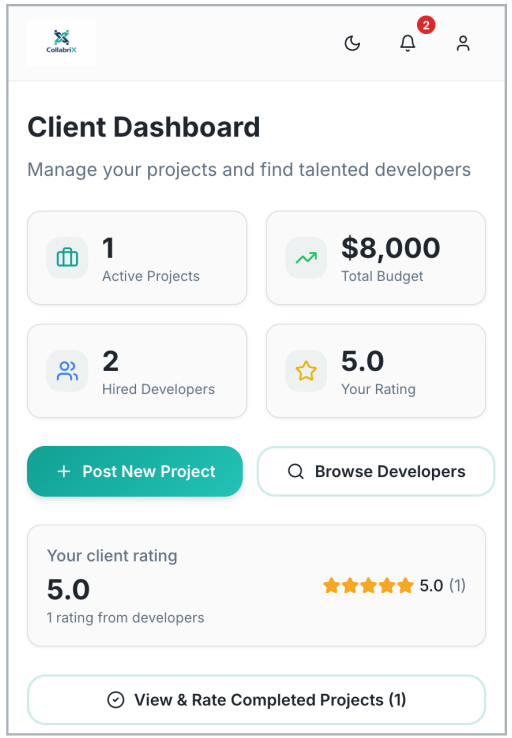
Team Formation Engine & Client Dashboard

Team Engine & Client Management

Figure 4.5 from Project Report



(a) Team Formation Engine



(b) Client dashboard

Interface Features & Highlights

Live Production Build Validation

- **Screen (a): Team Engine:** Interactive squad creator showing selected members, domain selector, live 67% synergy score badge, and missing technology alerts.
- **Reactive Skill Gap Detection:** When members are added, the UI instantly highlights missing skills (e.g. AWS, UI/UX), guiding the developer to recruit complementary peers.
- **Connection-Gated Squads:** Members can only be selected from accepted connections; BEFORE triggers reject unauthorized additions.
- **Screen (b): Client Dashboard:** Dedicated workspace for clients to post new projects, review applicants, and monitor active deliverables.
- **Project Budget & Complexity:** Form validated with Zod schemas enforcing valid budget numbers and standardized complexity tiers (Easy, Medium, Hard).

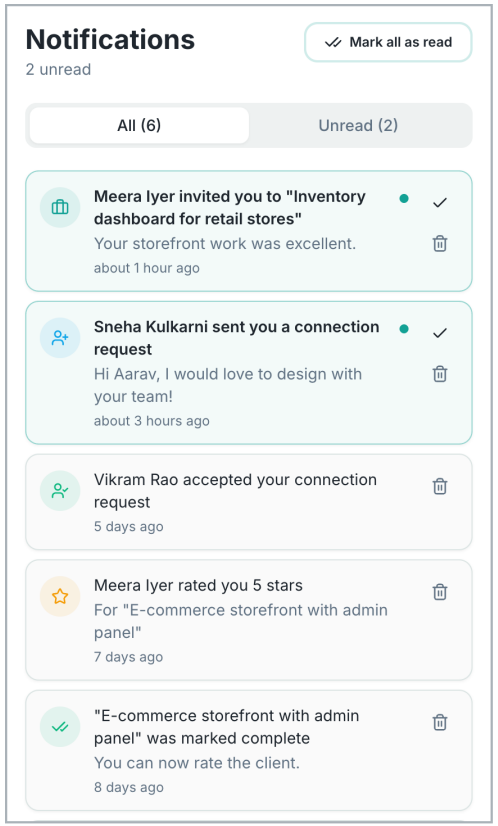
SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 21 illustrates the Team Formation Engine and Client Dashboard from Figure 4.5. On the left, Screen (a) shows a developer constructing a Team Batch for a Fullstack Web App. As teammates are toggled, the synergy score updates reactively to 67%, and the interface flags missing technologies like AWS and UI/UX. On the right, Screen (b) displays the Client Dashboard, providing project founders with an overview of active projects, budget allocations, candidate invitations, and direct access to project creation modals.

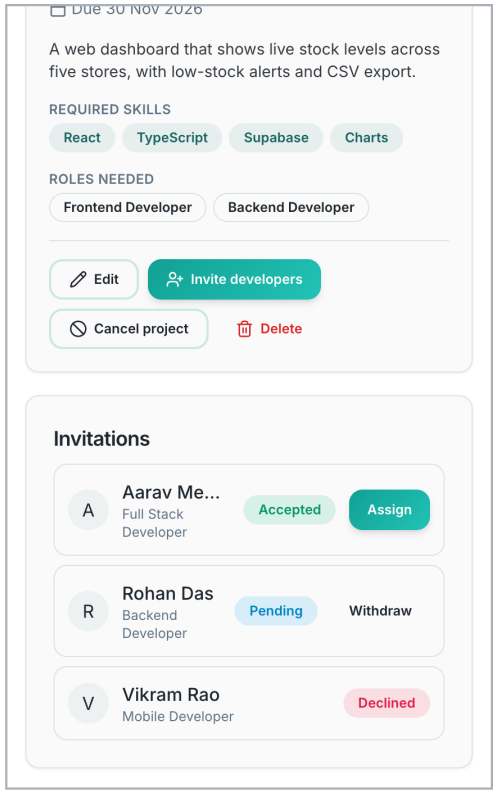
Notifications & Project Tracking Interface

Notifications & Project Assignment

Figure 4.6 from Project Report



(a) Notifications



(b) Project details (client)

Interface Features & Highlights

Live Production Build Validation

- **Screen (a): Notification Feed:** Unified chronological stream of trigger-generated alerts featuring actor avatars, time-ago stamps, and unread highlight dots.
- **One-Click Mark All Read:** Invokes optimistic mutation via TanStack Query, immediately clearing unread state in UI while dispatching database updates.
- **Screen (b): Project Details:** Client view of project scope, budget ($1,200), timeline, and candidate invitation tracking table.
- **Gated Assign Button:** Enforces Algorithm 3: the 'Assign' button remains disabled until the invited developer accepts the invitation in their own dashboard.
- **Role Authorization Isolation:** Developers viewing this page see a sanitized proposal view, while project owners see management controls.

SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 22 presents our Notifications Center and Project Tracking interface from Figure 4.6. On the left, Screen (a) shows the live Notification Feed, displaying event alerts with user avatars and relative timestamps. Clicking any notification marks it as read and navigates directly to the associated project or message thread. On the right, Screen (b) illustrates the Client Project Details view. Notice that a developer has been invited; the invitation status is tracked as 'pending', and the 'Assign' button remains disabled until the invited developer explicitly accepts, visually confirming that our database-enforced lifecycle rules are active.

Test Results: 104 DB Checks & Quality Metrics

104 / 104
Database Checks Passed
100% Pass Rate across 11 migrations

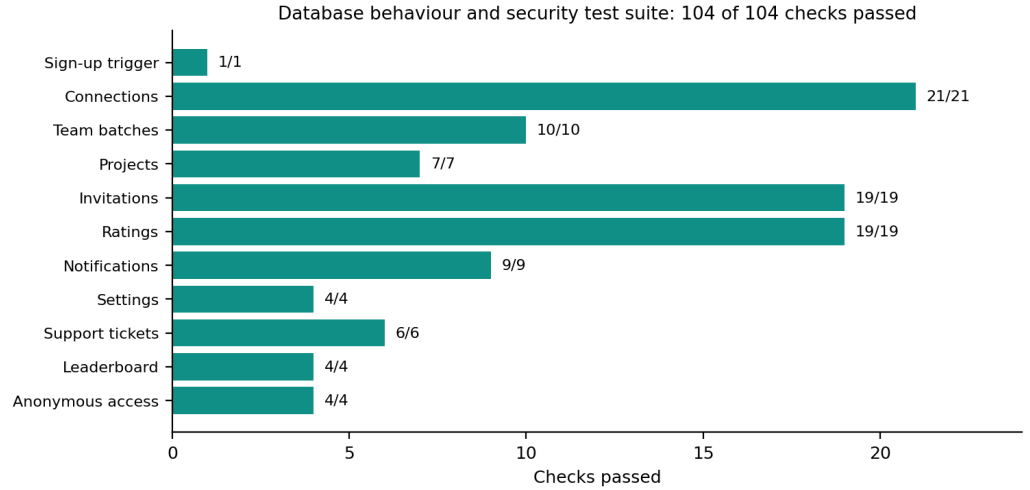
18 / 18
UI Browser Tests Passed
Puppeteer headless Chrome test suite

0 Errors
TypeScript Compilation
Strict npx tsc --noEmit check

49 → 2
ESLint Findings Reduced
14 errors & 35 warnings resolved

Database Test Suite Results (104 Checks)

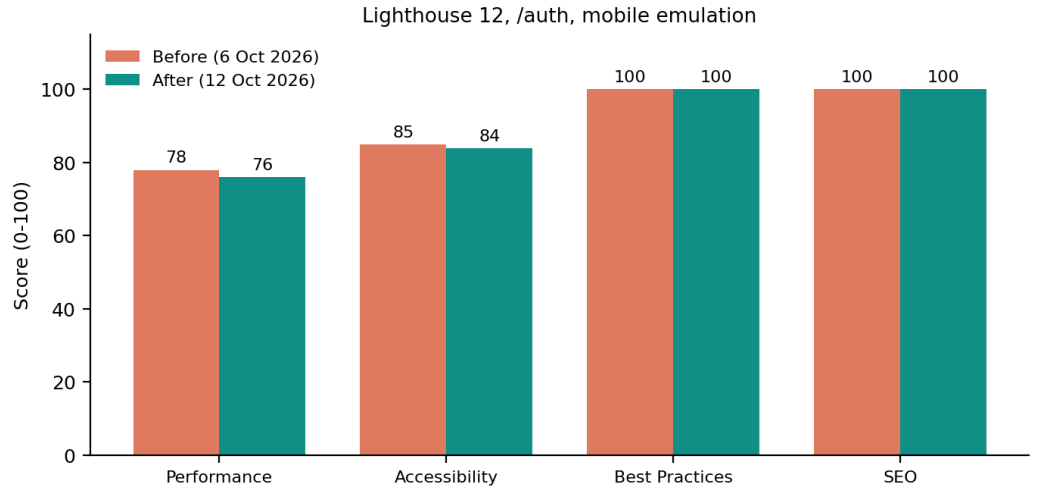
Figure 4.10 from Project Report



Coverage: Connections (21/21), Team Batches (10/10), Projects (7/7), Invitations (19/19), Ratings (19/19), Notifications (9/9), Settings (4/4), Support (6/6), Leaderboard (4/4), Anonymous (4/4), Sign-up (1/1).

Google Lighthouse Mobile Audit Scores

Figure 4.9 from Project Report



Lighthouse Metrics: Performance: 76 · Accessibility: 84 · Best Practices: 100 · SEO: 100. Audited with Lighthouse 12 on mobile emulation (simulated slow 4G, CPU throttling). Compressed bundle: 411 KB.

SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 23 delivers our empirical quality assurance and verification benchmarks, validating the reliability of CollabriX. First, we built an automated Python test suite that executed all 104 database integrity checks against PostgreSQL 16. As shown in Figure 4.10, every single check passed—validating connections, team batch constraints, invitation gating, completion-gated ratings, and anonymous RLS restrictions. Second, 18 automated Puppeteer browser tests verified authentication and navigation flows. Third, our codebase compiles with zero TypeScript errors, and we reduced ESLint findings from 49 down to just 2 minor development warnings. Finally, an independent Google Lighthouse mobile audit awarded CollabriX perfect 100/100 scores in Best Practices and SEO, with 84 in Accessibility and 76 in Performance on a 411 KB bundle.

Limitations & Future Work

■■ Current Prototype Boundaries

Table 5.2 from Project Report

• **Curated Job Listings:** Projects currently originate from verified on-platform postings rather than automated external job scraping APIs.

• **Relational Filter Matching:** Developer and project search uses SQL .ilike() and .contains() filters rather than semantic AI vector embeddings.

• **Text-Only Messaging:** Real-time chat supports text communication; multimedia attachments, voice notes, and video calling are not yet integrated.

• **Simulated Financial Escrow:** Payments are simulated through contract states without live Stripe escrow milestone disbursements in this academic phase.

• **In-App Notifications Only:** Notifications are currently displayed in-app; external transactional email and SMS dispatchers remain unconfigured.

■ Planned Future Enhancements

Strategic Evolution Roadmap

• **AI-Powered Team Matching:** Deploy embedding models to recommend complementary teammates based on GitHub commit histories and project domains.

• **Automated GitHub Verification:** OAuth integration with GitHub API to verify developer repositories, languages, and technical pull requests automatically.

• **WebRTC Audio & Video Conferencing:** Native peer-to-peer video calling for sprint standups, project kickoff meetings, and client interviews.

• **Stripe Milestone Escrow:** Smart financial escrow contracts releasing payments automatically upon client milestone sign-off.

• **React Native Mobile App:** Cross-platform mobile application compiled for iOS and Android with background push notifications.

SPEAKER SCRIPT / PRESENTATION NOTES:

Slide 24 provides an honest appraisal of our current prototype limitations alongside our future development roadmap. Presently, job postings are curated, search uses relational filters rather than semantic embeddings, chat is text-only, and payment escrow is simulated through project lifecycle states. In our future roadmap, we plan to implement AI-driven team recommendations using vector similarity over GitHub profiles, integrate automated GitHub repository verification, embed WebRTC for native video standups, implement Stripe escrow payments with milestone disbursements, and build a native React Native application for mobile push notifications.

PROJECT SUMMARY & OPEN FLOOR

Conclusion & Discussion

CollabriX: Proving the Power of Team-First Freelance Collaboration

KEY RESEARCH & ENGINEERING ACCOMPLISHMENTS

- **Team-First Shift:** Transformed freelance software delivery from isolated solo bidding to collaborative squad teaming.
- **Relational Integrity:** PostgreSQL Row-Level Security on all 12 tables prevents review fraud, state forgery, and unauthorized writes.
- **Algorithmic Synergy:** Algorithm 1 matches team capacity against domain standards, proactively alerting squads to skill gaps.
- **Verified Reputation:** Algorithm 2 computes open meritocratic leaderboard rankings exclusively from completed client deliveries.

SPECIAL ACADEMIC ACKNOWLEDGEMENTS

- **Project Guide:** Assistant Prof. Barkha Makwana for invaluable mentorship, architectural feedback, and rigorous evaluation.
- **Academic Institution:** Department of Computer Science & Engineering, Parul Institute of Engineering & Technology, Parul University.
- **Peer Community:** Fellow student developers whose collaboration workflows shaped our user personas and requirements.

THANK YOU! QUESTIONS & DISCUSSION WELCOME

SPEAKER SCRIPT / PRESENTATION NOTES:

In conclusion, CollabriX successfully proves that transitioning freelance marketplaces from solitary bidding queues to an architectural team-first paradigm empowers developers to deliver complex software while guaranteeing fair, verified reputation tracking. By enforcing business governance directly in PostgreSQL through Row-Level Security and stored procedures, we have built a platform that is secure by design and verified by 104 passing database tests and top Lighthouse scores. We express our sincere gratitude to our project guide, Assistant Professor Barkha Makwana, and the Department of Computer Science and Engineering at Parul University for their continuous encouragement and technical guidance. Thank you for your attention, and we are now pleased to open the floor to your questions and discussion.